

Universidad Autónoma de Santo Domingo

PRIMADA DE AMÉRICA / Fundada el 28 de octubre de 1538

Presentación

Facultad

Ciencias

Escuela

Escuela de informática

Asignatura

Lab Lenguaje de Programación III (INF 5170)

Tema

Resumen de AJAX y su uso

Estudiante

Cristopher Santiago Helena Perez

Matrícula

100658882

Profesor/a

Radhames Silverio Gónzalez

Fecha

29/10/2026

Índice

Introducción 3

1. ¿Qué es AJAX? 4

2. ¿Cuál es el propósito de AJAX? 4

3. ¿Cómo funciona AJAX en una página web? 4

4. Relación entre AJAX, JavaScript, el navegador y el servidor 5

5. Ventajas y desventajas del uso de AJAX 6

Ventajas: 6

Desventajas: 6

6. Usos actuales de AJAX en aplicaciones web 6

7. Diferencia entre una página web tradicional y una página que utiliza AJAX 7

8. Ejemplo de un documento AJAX y explicación de sus partes 8

Explicación del funcionamiento del ejemplo: 8

Conclusión 9

Referencias Bibliográficas 9

Introducción

Durante los primeros años de la World Wide Web, la interacción entre los usuarios y las aplicaciones alojadas en servidores se basaba en un modelo estrictamente sincrónico y tradicional heredado de los documentos de hipertexto estáticos. Cada vez que un usuario realizaba una acción tan simple como enviar un formulario, aplicar un filtro o hacer clic en un enlace de navegación, el navegador web se veía obligado a empaquetar una petición completa, enviarla al servidor y esperar a que este procesara la página entera para devolver un nuevo documento HTML. Este proceso provocaba el conocido e incómodo parpadeo de pantalla (*flash* blanco de recarga), consumía un ancho de banda considerable y ralentizaba drásticamente la fluidez de la experiencia de usuario, haciendo que las aplicaciones web parecieran lentas y toscas en comparación con el software de escritorio nativo.

La gran revolución tecnológica que transformó la web estática en una plataforma de aplicaciones dinámicas e interactivas vino de la mano de **AJAX** (*Asynchronous JavaScript and AJAX*). Acuñado formalmente a principios de la década del 2000, AJAX no introdujo un lenguaje de programación nuevo, sino que agrupó y potenció tecnologías preexistentes del lado del cliente (como JavaScript, el modelo de objetos del documento DOM y las peticiones de red asíncronas) para lograr un objetivo sin precedentes: permitir que una página web se comunicara con el servidor en segundo plano de manera silenciosa y transparente. Gracias a esta técnica, las aplicaciones web modernas pasaron de ser catálogos rígidos de páginas completas a convertirse en interfaces fluidas, reactivas y en tiempo real, sentando las bases conceptuales para el desarrollo de los frameworks actuales como React, Angular y Vue.

Informe sobre AJAX

1. ¿Qué es AJAX?

AJAX (*Asynchronous JavaScript and AJAX*) es un conjunto de técnicas de desarrollo web integradas que permite a las aplicaciones web enviar datos a un servidor y recibirlos de forma asíncrona en segundo plano, sin interferir con la visualización ni con el comportamiento de la página actual.

- No constituye un lenguaje de programación independiente, sino un enfoque arquitectónico que combina tecnologías nativas del navegador: **HTML y CSS** para la presentación visual, **el Modelo de Objetos del Documento (DOM)** para la manipulación dinámica de la interfaz, **JavaScript** como el lenguaje encargado de orquestar la lógica, y APIs de red nativas como fetch() o el objeto tradicional AJAXHttpRequest para la transferencia de datos.
- A pesar de que su acrónimo incluye "AJAX", en la práctica moderna **AJAX ha sido casi en su totalidad reemplazado por JSON** debido a su ligereza, aunque el término "AJAX" se mantiene como el estándar conceptual para definir peticiones asíncronas de cliente a servidor.

2. ¿Cuál es el propósito de AJAX?

El propósito fundamental de AJAX es **romper con el modelo de navegación sincrónico de página completa**, permitiendo actualizaciones parciales y dinámicas de la interfaz de usuario. Sus objetivos clave comprenden:

- **Mejorar radicalmente la experiencia de usuario (UX):** Eliminar los tiempos muertos y los parpadeos visuales por recarga de página, logrando que las aplicaciones web respondan de manera instantánea, similar a un programa de escritorio.
- **Optimizar el consumo de ancho de banda:** Evitar la transferencia redundante de código HTML, hojas de estilo y recursos gráficos enteros, transmitiendo estrictamente los datos puros necesarios (en formato JSON) para actualizar la vista.
- **Desacoplar la interfaz de la lógica de servidor:** Permitir que el navegador procese transacciones complejas, validaciones y renderizado de componentes de manera fluida y sin bloquear al usuario.

3. ¿Cómo funciona AJAX en una página web?

El ciclo de vida y funcionamiento de una operación AJAX se desarrolla a través de las siguientes fases secuenciales:

1. **Disparador de Evento:** El usuario realiza una acción en la interfaz web (por ejemplo, hace clic en un botón, selecciona una opción en un menú desplegable o escribe en un campo de texto).
2. **Creación de la Petición Asíncrona:** JavaScript intercepta dicho evento y genera una solicitud HTTP utilizando las APIs del navegador (como fetch).
3. **Envío en Segundo Plano:** El navegador transmite la petición al servidor web a través de la red de manera asíncrona. Durante este lapso, el usuario puede seguir interactuando libremente con la página web sin que la interfaz se congele.
4. **Procesamiento en el Servidor:** El servidor recibe la petición HTTP, ejecuta la lógica de backend correspondiente (consultas a bases de datos, validaciones o procesamiento de negocio) y genera una respuesta estructurada (típicamente en formato JSON).
5. **Recepción y Actualización del DOM:** El navegador recibe la respuesta del servidor. JavaScript captura los datos mediante funciones de devolución de llamada (*callbacks* o promesas) y modifica de manera dinámica y parcial los elementos específicos del DOM de la página reflejando los cambios de forma instantánea.

4. Relación entre AJAX, JavaScript, el navegador y el servidor

Para comprender la sinergia operativa de AJAX, es necesario analizar el rol de cada componente tecnológico involucrado:

- **El Navegador Web:** Actúa como el entorno de ejecución del cliente. Proporciona el motor de JavaScript y expone las interfaces de programación de red (como fetch y XMLHttpRequest) que permiten abrir canales de comunicación HTTP.
- **JavaScript:** Funciona como el director de orquesta. Es el lenguaje responsable de detectar las interacciones del usuario, construir y enviar las peticiones asíncronas, y procesar la respuesta del servidor para inyectarla en la interfaz.
- **El Servidor:** Es el sistema remoto encargado de escuchar las peticiones HTTP entrantes, descifrar los parámetros enviados, procesar las reglas de negocio y devolver una respuesta estructurada con los datos solicitados.
- **AJAX:** Es el puente metodológico y conceptual que une al navegador, a JavaScript y al servidor mediante la ejecución de llamadas asíncronas que evitan la recarga completa del documento.

5. Ventajas y desventajas del uso de AJAX

Ventajas:

- **Interactividad y Fluidez Superior:** Ofrece una experiencia de usuario sumamente ágil, moderna y sin interrupciones visuales.
- **Eficiencia en el Ancho de Banda:** Al transferir únicamente datos estructurados y no páginas web completas, se reduce drásticamente el volumen de tráfico de red.
- **Procesamiento Asíncrono No Bloqueante:** El usuario no tiene que esperar a que el servidor termine de procesar la tarea para seguir utilizando la interfaz de la aplicación.
- **Modularidad:** Permite actualizar secciones concretas de un sitio web (como una tabla de resultados o un contador de carrito) de forma independiente.

Desventajas:

- **Complicación con el Historial del Navegador:** Al no recargarse la página, las acciones internas ejecutadas mediante AJAX no actualizan por defecto la barra de direcciones ni registran correctamente los botones de "Atrás" y "Adelante" del navegador (requiriendo APIs adicionales como HTML5 History API).
- **Desafíos de Indexación SEO:** Los motores de búsqueda tradicionales pueden tener dificultades para rastrear e indexar contenido que se carga dinámicamente en segundo plano si no se implementan estrategias avanzadas de renderizado en el servidor (*SSR*).
- **Vulnerabilidades de Seguridad:** Una mala configuración de las políticas de origen cruzado (*CORS*) o la falta de validación de datos en el cliente y servidor pueden exponer la aplicación a ataques informáticos.

6. Usos actuales de AJAX en aplicaciones web

En el desarrollo web contemporáneo, AJAX es omnipresente y se aplica en múltiples escenarios:

- **Validaciones de Formularios en Tiempo Real:** Comprobación instantánea de disponibilidad de nombres de usuario o correos electrónicos mientras se escribe en un formulario de registro.
- **Sugerencias de Búsqueda Predictiva (*Autocompletado*):** Despliegue dinámico de resultados y listas desplegables de productos mientras el usuario introduce caracteres en un buscador.
- **Carga Infinita y Paginación Dinámica:** Descarga y renderizado automático de más elementos multimedia o productos al llegar al final de la pantalla (habitual en redes sociales y e-commerce).

- **Actualización de Paneles de Control (*Dashboards*):** Refresco automático de métricas financieras, gráficos estadísticos y notificaciones sin intervención manual.

7. Diferencia entre una página web tradicional y una página que utiliza AJAX

Criterio	Página Web Tradicional (Sincrónica)	Página Web con AJAX (Asincrónica)
Modelo de Comunicación	Sincrónico: Cada acción requiere una solicitud y respuesta de página completa.	Asincrónico: Solicitudes silenciosas en segundo plano sin interrumpir al usuario.
Experiencia Visual	Se produce un parpadeo de pantalla (*flash* blanco) cada vez que el usuario interactúa.	La interfaz permanece estática y fluida; solo se actualizan las secciones afectadas.
Velocidad de Carga Percibida	Lenta, ya que depende de la descarga completa de nuevos recursos HTML y CSS.	Instantánea, debido a que solo se transfieren los datos ligeros necesarios.
Uso de Recursos de Red	Alto, al descargar documentos y estructuras completas repetidamente.	Mínimo y optimizado, transfiriendo exclusivamente cargas de datos en formato JSON.

8. Ejemplo de un documento AJAX y explicación de sus partes

El siguiente código implementado en JavaScript moderno utilizando la API fetch demuestra cómo funciona una petición AJAX aplicada al e-commerce **AbbaTech** para consultar el precio de un producto en tiempo real sin recargar la página:

```
// Función que simula una petición AJAX asíncrona al servidor de AbbaTech
function consultarPrecioProducto(codigoProducto) {
    console.log("Enviando petición asíncrona al servidor en segundo plano...");

    // Se ejecuta la llamada AJAX mediante la API fetch
    fetch(`https://api.abbatech.com/productos/${codigoProducto}`)
        .then(response => {
            // Verificamos si la respuesta del servidor es correcta
            if (!response.ok) {
                throw new Error("Producto no encontrado en el inventario.");
            }
            return response.json(); // Convertimos la respuesta cruda a un objeto JS
        })
        .then(data => {
            // Actualizamos de manera dinámica el DOM de la página sin recargarla
            document.getElementById("nombre-producto").textContent = data.nombre;
            document.getElementById("precio-producto").innerText = `$${data.precio}`
            console.log("¡Interfaz actualizada exitosamente mediante AJAX!");
        })
        .catch(error => {
            console.error("Error al procesar la petición AJAX:", error);
            document.getElementById("mensaje-error").textContent = error.message;
        });
}
```

Explicación del funcionamiento del ejemplo:

1. Al invocar la función consultarPrecioProducto('PROD-01'), JavaScript dispara una petición HTTP al servidor de forma asíncrona utilizando fetch.
2. El navegador web del usuario **no se bloquea ni se recarga**; el usuario puede seguir explorando la tienda o haciendo clic en otros botones de la interfaz de AbbaTech.
3. Una vez que el servidor procesa la consulta y responde con los datos en formato JSON, la promesa se resuelve (.then()).
4. El script captura los datos recibidos y modifica directamente las etiquetas HTML correspondientes (#nombre-producto y #precio-producto) mediante la manipulación del DOM, logrando que el precio del artículo se refleje instantáneamente en la pantalla del usuario.

Conclusión

En conclusión, el desarrollo y la implementación generalizada de **AJAX** marcaron un antes y un después en la historia de la ingeniería de software orientada a la web. Al dotar a las páginas tradicionales de la capacidad de comunicarse de manera asíncrona y silenciosa con los servidores, AJAX erradicó los molestos tiempos muertos y las recargas completas de pantalla, elevando la experiencia de usuario a estándares comparables con los de las aplicaciones de escritorio nativas. Su evolución conceptual sentó las bases para la arquitectura de las APIs RESTful modernas y el desarrollo de potentes interfaces dinámicas basadas en componentes. Hoy en día, comprender el funcionamiento de las peticiones asíncronas es un requisito indispensable y transversal para cualquier desarrollador que aspire a construir sistemas web eficientes, altamente escalables y verdaderamente interactivos.

Referencias Bibliográficas

 Garrett, J. J. (2005). *Ajax: A New Approach to Web Applications*. Adaptive Path. Recuperado el 31 de octubre de 2026, de https://adaptivepath.org/ideas/ajax-new-approach-to-web-applications/

 Mozilla Developer Network (MDN). (2025). *Guía de AJAX y Fetch API*. Mozilla Contributors. Recuperado el 31 de octubre de 2026, de https://developer.mozilla.org/es/docs/Web/Guide/AJAX

 W3Schools. (s. f.). *AJAX Introduction*. Refsnes Data. Recuperado el 31 de octubre de 2026, de https://www.w3schools.com/xml/ajax_intro.asp

 GeeksforGeeks. (2024). *AJAX Working and Applications in Web Development*. GeeksforGeeks Data structures. Recuperado el 31 de octubre de 2026, de https://www.geeksforgeeks.org/what-is-ajax-web-development/